

MS-CS Annual Assessment Report

- **Program Names:** Master of Computer Science (MCS) and Master of Science in Computer Science (MSCS)
- **Department Name:** Computer Science
- **College Name:** College of Computing
- **Faculty Assessment Coordinator:** CS Graduate Committee
- **Assessment Cycle:** Academic Year 2024–25

Note on Combined Assessment

The **MCS** and **MSCS** programs share largely overlapping student populations, faculty, and curriculum. The program learning goals for both degrees are identical in practice, and the same core courses are used to assess student performance. For this reason, we are reporting **a single, unified assessment** for both programs. Differences between the programs (e.g., thesis option in MSCS, professional orientation of MCS) are acknowledged in curriculum, but their assessment outcomes are aligned.

1. Program Learning Goals for 2024–25

- **G1.** Have the ability to analyze, design, and implement algorithmic approaches.
- **G2.** Have the ability to analyze, design, and implement software and computer systems.
- **G3.** Have the ability to write complex computer programs using modern programming paradigms.

2. Assessment Methodology

Learning Goal	Sample Size (Enrollment)	Semester Collected	Rubric Used	Artifact Source (Course/Instructor/Assignment/Other)	Month/Year of Evaluation	Evaluator(s)
G1: Algorithmic Approaches	26	Spring 2025	Algorithmic Technique Rubric	CS535 – Design and Analysis of Algorithms (Instructor: Peng-Jun Wan)	May 2025	Sanjiv Kapoor
G2: Software & Systems	9	Spring 2025	Systems Rubric	CS550 – Advanced Operating Systems (Instructor: Xian-He Sun)	May 2025	Oliver Michel
G3: Programming	138	Spring 2025	Programming Rubric	CS525 – Advanced Database Organization (Instructor: Gerald Balekaki)	May 2025	Yanxue Jia

3. Assessment Results

Learning Goal G1: Algorithmic Approaches

Results:

- Average student performance across topics = **80.5% meeting standard**.
- Strongest areas: Shortest paths, dynamic programming, approximation algorithms (>80%).
- Weaker area: Minimum spanning tree and arborescence (72%).
- Overall, more than 4/5 students achieved expected proficiency.

Discussion: Many students entered without sufficient background in algorithms (CS430 equivalent). Homework 0 was used to diagnose gaps, and weaker students were encouraged to review prerequisites. Advanced topics were adapted to match preparation levels. Recommendation: Apply **recitation sessions** for additional support.

Learning Goal G2: Software & Systems

Results:

- Student achievement rates: 87%–95% across assignments, projects, and final exam.
- Strongest performance: Programming assignments implementing centralized and distributed systems (~92–95%).
- Overall mastery of the field: 87% (via final exam).

Discussion: Students demonstrated solid ability to design and implement distributed system components, with strong results in both theory and practice. The results indicate good preparation for advanced systems work. Recommendation: Consider expanding projects to include **modern cloud/distributed frameworks** for greater alignment with industry practice.

Learning Goal G3: Programming

Results:

- Overall success rate: **94.5% of students meeting/exceeding expectations**.
- Assignment-specific performance: >92% across buffer management, B⁺-tree indexing, and advanced data access layers; >95% in file I/O and memory management.

Discussion: Despite the large class size (~138 students), the course structure of group assignments (3–4 students each) ensured consistent participation. Results show students can write complex programs using modern paradigms. Recommendation: Keep current assignment structure, but **incorporate modern NoSQL systems** (e.g., key-value/document stores, clustering, replication) into future assignments to broaden student exposure.

4. Discussion of Findings

- **G1:** Students demonstrated strong achievement overall, though some gaps remain in mathematical and algorithmic preparation.
- **G2:** Strong performance in systems programming assignments; students are meeting outcomes with high proficiency.

- **G3:** Students exceeded expectations in programming, with recommendations to expand curriculum toward NoSQL and distributed data systems.

5. Improvement Plans

Planned Modification	Responsible Person	Implementation Date	Intended Outcome
Introduce recitation/tutorial support for CS535	CS535 Instructor & Graduate Committee	AY25–26	Strengthen preparation in algorithmic analysis
Expand CS550 projects with modern distributed/cloud frameworks	CS550 Instructor	AY25–26	Improve applicability to current systems challenges
Revise CS525 assignments to include NoSQL-style projects	CS525 Instructor	AY25–26	Broaden student programming experience to modern database paradigms

6. Assessment Process Recommendations

- Diagnostic assignments (e.g., Homework 0 in CS535) are effective and should be extended.
- Consider collecting **direct project-based evidence** earlier in the programs.
- For large enrollment courses (CS525), continue group work but enhance tracking of individual contributions.

7. Proposed Assessment Plan for Next Year

Learning Goal	Expected Sample Size	Semester(s) Collected	Rubric Used	Artifact Source	Evaluation Date	Evaluator(s)
G1	TBD	AY25–26	Algorithmic Technique Rubric	CS535	TBD	TBD
G2	TBD	AY25–26	Systems Rubric	CS550	TBD	TBD
G3	TBD	AY25–26	Programming Rubric	CS525	TBD	TBD

8. Appendices

Appendix A: Rubric for Objective G1

- **Course:** CS535 – Design and Analysis of Algorithms
- **Instructor:** Peng-Jun Wan
- **Evaluator:** Sanjiv Kapoor
- **Semester:** Spring 2025
- **Enrollment:** 26 students

Criteria / Topic	Description	Weight	Measurement Method	Performance Target	% Students Achieving
Constrained graph traversal	Graph construction and replication, inference via reachability	10%	Written assignment ≥ 80 OR quiz/midterm/final ≥ 70	$\geq 70\%$	78%
Shortest paths, widest paths, min-mean circuits	Greedy/dynamic programming, relaxation, reweighting, boundary between P and NP	10%	Written assignment ≥ 80 OR quiz/midterm/final ≥ 70	$\geq 70\%$	84%
Backtracking with stack	Linear time with processing, backward elimination, amortized analysis	10%	Written assignment ≥ 80 OR quiz/midterm/final ≥ 70	$\geq 70\%$	82%
Speed-up in dynamic programming	Monge and submodularity, speedup by monotonicity, charging argument	10%	Written assignment ≥ 80 OR quiz/midterm/final ≥ 70	$\geq 70\%$	82%
Decrease and conquer with median	Parametric search, median evaluation, contraction of instances	10%	Written assignment ≥ 80 OR quiz/midterm/final ≥ 70	$\geq 70\%$	82%
Minimum spanning tree and arborescence	Hybrid algorithms, contraction and expansion	10%	Quiz/midterm/final ≥ 70	$\geq 70\%$	72%
Bipartite matching and covering	Potential method, augmenting in batch for speedup	10%	Written assignment ≥ 80 OR quiz/midterm/final ≥ 70	$\geq 70\%$	80%
Max-flow, min-cut, and applications	Duality, pseudo-polynomial, weakly/strongly polynomial, relaxation	15%	Written assignment ≥ 80 OR quiz/midterm/final ≥ 70	$\geq 70\%$	80%
Approximation algorithms	Restriction, relaxation, scaling, primal-dual methods	15%	Written assignment ≥ 80 OR quiz/midterm/final ≥ 70	$\geq 70\%$	83%

Overall Performance:

- **Average achievement:** 80.5% of students met or exceeded expectations.
- **Strengths:** Students performed well on shortest paths, dynamic programming, and approximation algorithms.
- **Weaknesses:** Performance was weaker in MST/arborescence (72%).

Appendix B: Rubric for Objective G2

- **Course:** CS550 – Advanced Operating Systems
- **Instructor:** Xian-He Sun
- **Evaluator:** Oliver Michel
- **Semester:** Spring 2025
- **Enrollment:** 9 students

Criteria / Topic	Description	Weight	Measurement Method	Performance Target	% Students Achieving
Concepts of distributed systems	Understanding of distributed system principles	9%	3 written assignments (≥ 80)	$\geq 80\%$	92%
Centralized file sharing system	Design & implementation of centralized file sharing	14%	Programming Assignment 1 (≥ 80)	$\geq 80\%$	95%
Distributed file sharing system	Design & implementation of distributed file sharing	14%	Programming Assignment 2 (≥ 80)	$\geq 80\%$	92%
Consistency implementation	Implementing consistency protocols	14%	Programming Assignment 3 (≥ 80)	$\geq 80\%$	90%
Overall knowledge of the field	Demonstrated comprehensive OS knowledge	28%	Final exam (≥ 60)	$\geq 60\%$	87%
Research on selected topic	Research project and report	15%	Final project report (≥ 80)	$\geq 80\%$	91%

Overall Performance:

- Students achieved consistently high scores (87–95%) across assignments, projects, and exams.
- Strongest performance was in **centralized/distributed system implementations**.
- Improvement could be made by aligning projects more closely with modern **cloud/distributed frameworks**.

Appendix C: Rubric for Objective G3

- **Course:** CS525 – Advanced Database Organization
- **Instructor:** Gerald Balekaki
- **Evaluator:** Yanxue Jia
- **Semester:** Spring 2025
- **Enrollment:** 138 students

Criteria / Topic	Description	Weight	Measurement Method	Performance Target	% Students Achieving
File I/O & modular design	Low-level file I/O, error handling, modular system design	22%	Assignment #1 (passing grade)	≥ 70%	>95.5%
Buffer pool & memory management	Memory management, buffer pools, algorithm design	22%	Assignment #2 (passing grade)	≥ 70%	>92%
Structured data access & integration	Condition-based retrieval, layered database functionality	28%	Assignment #3 (passing grade)	≥ 70%	>98%
Data indexing structures	Implementation of B ⁺ -tree indexing system	28%	Assignment #4 (passing grade)	≥ 70%	>92.5%

Overall Performance:

- **94.5%** of students met or exceeded expectations.
- Assignments were group-based (3–4 students), with individual contributions monitored.
- Students demonstrated strong mastery of relational database systems.
- Future improvement: integrate **modern NoSQL-style systems** (e.g., key-value/document databases, replication, clustering) to broaden training.

Appendix D: Summary of Cumulative Changes from Prior Assessments

This appendix summarizes notable changes implemented in the MCS/MSCS programs over the past several years as a result of ongoing assessment and program review. The purpose is to demonstrate continuous improvement and responsiveness to identified student learning challenges.

1. Curriculum Adjustments

- **New degree programs and specializations** were introduced (e.g., Master of Artificial Intelligence, Master of Cybersecurity, Master of Data Science, CDSOR) in response to emerging areas in computing.
- **New core areas** were added to reflect evolving disciplinary needs (e.g., Networks and Security, Databases, Software Engineering, Computational Intelligence).
- **New courses** developed in response to assessment feedback and industry trends (e.g., CS553 Cloud Computing, CS554 Data-Intensive Computing, CS558 Advanced Computer Security, CS579 Online Social Network Analysis).

2. Course-Level Changes

- **CS535 (Algorithms):** Added diagnostic homework (Homework 0) to identify students lacking prerequisite knowledge; encouraged enrollment in CS430 before CS535.
- **CS550 (Operating Systems):** Expanded programming assignments from centralized to distributed file systems, incorporating consistency protocols. Plans for future alignment with cloud frameworks.
- **CS525 (Advanced Databases):** Assignment structure revised to progressively build a database system. Recommendations made to integrate NoSQL paradigms in future iterations.
- **Recitations/tutorials** were added in some core courses (modeled after CS430) to improve mathematical and algorithmic preparation.

3. Assessment and Evaluation Process

- **Rubrics** were standardized across core courses to ensure consistency in evaluating student learning outcomes.
- **Group assignments** in large-enrollment courses (e.g., CS525) incorporated individual contribution tracking to ensure fair assessment.
- **Faculty evaluation coordination** was formalized through area managers and the Graduate Committee.

4. Program Standards and Admissions

- **GRE minimums and admissions standards** were raised to improve baseline preparation of entering graduate students.
- **Diagnostic tools** (placement assignments, sample problems) were introduced to help incoming students self-assess and prepare.
- Increased emphasis on **student preparation in mathematics and logic** for success in theory- and language-focused courses.

5. Facilities and Resources

- Expanded **office and lab space** in response to program growth.
- Introduced **course fees for specialized equipment** to maintain lab-based courses.

6. Strategic Initiatives

- Development of a **departmental strategic plan** to align teaching, research, and curriculum with long-term institutional goals.
- Increased **targeted recruiting efforts** (e.g., India recruiting campaigns, scholarships) to diversify and strengthen the MSCS/MCS student body.
- Continuous **review of course syllabi** and **evaluation of non-core electives** to maintain program relevance.